Convolutional Neural Networks

A Compact Course

Oliver Benz

20 October 2026

Contents

1 Overview **2**

2 Module 1 — Motivation **3**

2.1 Images as tensors . . . 3
2.2 The problem with dense layers . . . 3
2.3 Two restrictions that turn a dense layer into a convolution . . . 3
2.4 Hierarchy: why stacking layers helps . . . 4
2.5 The symmetry view . . . 4
2.6 Where CNNs are used . . . 5

3 Module 2 — Neural network foundations **5**

3.1 Layers . . . 5
3.2 From outputs to probabilities . . . 5
3.3 The loss: cross-entropy . . . 6
3.4 Training with gradient descent . . . 6
3.5 Backpropagation: computing the gradient . . . 6
3.6 Generalisation . . . 7

4 Module 3 — Convolution math **7**

4.1 Convolution or cross-correlation? . . . 7
4.2 The general conv layer . . . 8
4.3 Stride . . . 8
4.4 Padding . . . 8
4.5 Output size . . . 8
4.6 Parameters and compute . . . 9
4.7 Receptive field . . . 9

4.8 1×1 convolutions 9

5 Module 4 — Backprop through conv and pooling 9

5.1 Setup 10

5.2 Weight and bias gradients 10

5.3 Input gradient 10

5.4 Nonlinearities and pooling 11

5.5 Check yourself 11

6 Module 5 — Building blocks and architectures 11

6.1 Batch normalisation 11

6.2 Dropout 12

6.3 Residual connections 12

6.4 Weight initialisation 13

6.5 Landmark architectures 13

6.6 The canonical layout 13

7 Module 6 — Application: training a CNN in PyTorch 13

7.1 Setup 14

7.2 Tensor shapes through the network 14

7.3 The full script (`train_cnn.py`) 14

7.4 What each part does 16

7.5 Reading the output 16

7.6 Using the trained model 17

8 Exercises and next steps 17

Overview

By the end of these six modules you will train a CNN on CIFAR-10 in PyTorch and understand every line of the math behind it. Each module is short: the idea, the equations, and what they mean in practice.

Prerequisites: linear algebra, multivariable calculus (chain rule, gradients), basic probability, and working Python. No prior ML is assumed.

Notation: scalars in italics, vectors bold lowercase, matrices/tensors bold uppercase. Indices: c = channel, (i, j) = spatial position, k = output channel / filter, n = sample in a batch.

Module	Topic	Outcome
1	Motivation	Why images need a special architecture
2	Neural network foundations	Loss, gradient descent, backprop
3	Convolution math	Shapes, parameters, receptive fields
4	Backprop through conv and pooling	Gradients are themselves convolutions
5	Building blocks and architectures	From LeNet to ResNet
6	Application in PyTorch	A complete, working training script

Module 1 — Motivation

A CNN is an ordinary neural network whose layers are restricted in two ways that match how images work. This module starts from a dense layer and shows how those two restrictions turn it into a convolution.

Images as tensors

An RGB image is a three-dimensional array $\mathbf{X}$ with entries $X_{c,i,j}$. Here $c \in \{1, 2, 3\}$ is the colour channel (red, green, blue), i is the row and j the column of the pixel. An image of height H and width W therefore consists of $3HW$ numbers.

The problem with dense layers

A dense (fully connected) layer connects every input to every output. Its output is a vector $\mathbf{h} = (h_1, \ldots, h_M)$ of M numbers, and each component h_m is called a **unit** (or neuron) of the layer. Written with the image indices, unit m computes

$$h_m = \sigma\Big(\sum_{c,i,j} W_{m,c,i,j}\, X_{c,i,j} + b_m\Big), \tag{1}$$

where $W_{m,c,i,j}$ is the weight connecting pixel (c, i, j) to unit m, b_m is a bias, and σ is a nonlinear function (Module 2). Each unit has its own weight for every pixel. This causes two problems.

- **Size.** A 224×224 RGB image has 150,528 inputs. A layer with 1,000 units therefore needs about 1.5×10^8 weights, which is slow to train and overfits easily.
- **No spatial structure.** The layer has no notion of neighbouring pixels: pixel $(10, 10)$ and pixel $(11, 10)$ are treated as unrelated as two pixels at opposite corners. If the object in the image moves by one pixel, it hits completely different weights, so a feature learned at one position has to be learned again at every other position.

Two restrictions that turn a dense layer into a convolution

Arrange the output units on a grid as well, so that unit (i, j) describes the image near pixel (i, j). Then impose two assumptions about images.

1. Locality. Edges, corners and textures are determined by a small neighbourhood of pixels. So unit (i, j) only receives weights for the pixels in a $K \times K$ window centred on pixel (i, j). K is called the kernel size. It is odd, typically 3, so the window reaches $r = (K - 1)/2$ pixels to

each side of the centre ($r = 1$ for $K = 3$). With offsets $u, v \in \{-r, \dots, r\}$ from the centre:

$$h_{i,j} = \sigma\Big(\sum_{c} \sum_{u=-r}^{r} \sum_{v=-r}^{r} W_{i,j,c,u,v}\, X_{c,\, i+u,\, j+v} + b_{i,j} \Big). \tag{2}$$

For $K = 3$ this sums over pixel (i, j) and its eight neighbours. Near the border the window sticks out of the image; Module 3 handles this with padding. The restriction reduces the weights per unit from $3HW$ to $3K^2$, but each position (i, j) still has its own set of weights.

2. Weight sharing. The statistics of an image are roughly the same everywhere (stationarity): an edge detector that is useful in the top-left corner is just as useful in the bottom-right. So every position uses the same weights, $W_{i,j,c,u,v} = w_{c,u,v}$ and $b_{i,j} = b$:

$$h_{i,j} = \sigma\Big(\sum_{c,u,v} w_{c,u,v}\, X_{c,\, i+u,\, j+v} + b \Big). \tag{3}$$

The sum is now a discrete convolution of the image with the small array w, called the **kernel** or **filter** (strictly a cross-correlation, see Module 3). The kernel is slid over the image, and at each position it produces one number that measures how well the patch matches the pattern stored in w. The resulting grid of numbers $h_{i,j}$ is a **feature map**.

One kernel detects one kind of pattern. A conv layer therefore uses C_{out} kernels in parallel and produces C_{out} feature maps, called the output **channels**. They play the same role for the next layer as the colour channels do for the first.

Parameter count. A kernel has one weight per input channel and window position, $C_{\text{in}}K^2$ weights, plus one bias. C_{in} is the number of input channels (3 for RGB). With C_{out} kernels, a layer has

$$N_{\text{params}} = C_{\text{out}}\,(C_{\text{in}}K^2 + 1) \tag{4}$$

parameters. For $K = 3$, $C_{\text{in}} = 3$ and $C_{\text{out}} = 64$ this is $64 \cdot (27 + 1) = 1{,}792$. The number does not depend on the image size $H \times W$, compared with 1.5×10^8 for the dense layer above.

Hierarchy: why stacking layers helps

A single 3×3 layer only sees a 3×3 patch of pixels: enough for an edge, not for an eye. But the output of a conv layer is again a grid with channels, so a second conv layer can be applied to it. Each unit of the second layer combines a 3×3 window of first-layer units, each of which saw 3×3 pixels, so it depends on a 5×5 pixel region.

The input region a unit depends on is its **receptive field**. It grows with every layer, and faster when layers downsample the image (Module 3). Deep layers can therefore combine edges into textures, textures into object parts, and parts into whole objects, while each layer stays local and small.

The symmetry view

Weight sharing has an exact consequence. Let T_a be the operator that shifts an image by the vector $a = (a_1, a_2)$, so that the shifted image at pixel (i, j) equals the original at $(i - a_1, j - a_2)$. Because the same kernel is used at every position, convolving a shifted image gives the shifted feature map, up to effects at the border. Writing $\star$ for the convolution:

$$(T_a X) \star w = T_a(X \star w). \tag{5}$$

This property is **translation equivariance**: when an object moves, its features move with it but do not change. A classifier needs **invariance** instead, because the label "cat" should not change at all when the cat moves. Pooling and averaging over positions (Module 5) turn the equivariant feature maps into an invariant output.

The general idea is to build a known symmetry of the data into the structure of the layer instead of learning it from examples. The same reasoning with other symmetry groups leads to equivariant networks for rotations or gauge symmetries.

Where CNNs are used

CNNs work on any data that lives on a regular grid and has local structure: image classification, object detection, segmentation, medical imaging, audio spectrograms, and board-game positions (AlphaGo's policy and value networks are CNNs over the 19×19 board).

Module 2 — Neural network foundations

A neural network is a function f_θ with adjustable parameters θ, the collection of all its weights and biases. Training means choosing θ so that f_θ gives the right answers on example data. This module defines the function, the loss that measures how wrong an answer is, and how the gradient of the loss is computed. Everything here applies unchanged to CNNs, because a conv layer is just a layer with a particular weight structure.

Layers

A layer takes a vector of inputs $\mathbf{a}$, applies an affine map, and then applies a nonlinear function σ to each component. A network stacks L such layers. Its input is $\mathbf{a}^{(0)} = \mathbf{x}$, and layer $l = 1, \dots, L$ computes

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma\big(\mathbf{z}^{(l)}\big). \tag{6}$$

$\mathbf{W}^{(l)}$ is the weight matrix and $\mathbf{b}^{(l)}$ the bias vector of layer l. $\mathbf{z}^{(l)}$ is called the pre-activation and $\mathbf{a}^{(l)}$ the activation.

The nonlinearity is essential. A composition of affine maps is again affine, so without σ a network of any depth could only represent a linear function. The standard choice is the ReLU (rectified linear unit), $\sigma(z) = \max(0, z)$. Its derivative is $\sigma'(z) = 1$ for $z > 0$ and 0 otherwise. It is cheap, and unlike sigmoid or tanh it does not flatten out for large inputs, which keeps gradients from vanishing.

In a CNN the affine map of a conv layer is the convolution from Module 1. It is still linear in the input, just with a sparse, repeated weight matrix.

From outputs to probabilities

For classification into C classes, the last layer has C units. Their raw values $z_1, \dots, z_C$ are called **logits**: a larger logit means the model considers that class more likely. The **softmax** function turns them into probabilities that are positive and sum to 1:

$$p_c = \frac{e^{z_c}}{\sum_{c'=1}^{C} e^{z_{c'}}}. \tag{7}$$

This is a Boltzmann distribution with energies $-z_c$ at temperature 1, and the denominator is the partition function.

The loss: cross-entropy

Let y be the index of the correct class. A good model assigns a high probability p_y to it, so the loss is the negative logarithm of that probability:

$$\mathcal{L} = -\log p_y = -z_y + \log \sum_{c'} e^{z_{c'}}. \tag{8}$$

$\mathcal{L}$ is 0 when $p_y = 1$ and grows without bound as $p_y \to 0$. Minimising the summed loss over a training set is the same as maximising the probability the model assigns to the observed labels, so this is maximum-likelihood estimation. The name **cross-entropy** comes from writing the target as a one-hot vector $\mathbf{y}$ ($y_c = 1$ for the correct class, 0 otherwise), which gives $\mathcal{L} = -\sum_c y_c \log p_c$, the cross-entropy between the target and predicted distributions.

Differentiating the second form with respect to a logit gives a remarkably simple result:

$$\frac{\partial \mathcal{L}}{\partial z_c} = p_c - y_c. \tag{9}$$

The gradient is predicted probability minus target probability. Gradient descent therefore raises the logit of the correct class and lowers every other logit in proportion to how much probability it took. This vector is where backpropagation starts.

Training with gradient descent

The training set consists of N pairs $(\mathbf{x}_n, y_n)$ of inputs and correct labels. Training minimises the mean loss

$$J(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}\big(f_\theta(\mathbf{x}_n),\, y_n\big) \tag{10}$$

by repeatedly taking a small step against its gradient. Computing the exact gradient over all N examples for every step is too expensive, so **stochastic gradient descent** (SGD) estimates it from a random mini-batch B of typically 32–256 examples:

$$\theta \leftarrow \theta - \eta \, \frac{1}{|B|} \sum_{n \in B} \nabla_\theta \mathcal{L}_n. \tag{11}$$

The step size η is the **learning rate**, and $\mathcal{L}_n$ is the loss on example n. One pass through the whole training set is an **epoch**. Two common improvements: **momentum** keeps a running average of past gradients and steps along it, which smooths out the noise of individual batches; **Adam** additionally scales each parameter's step by a running estimate of its gradient magnitude.

Backpropagation: computing the gradient

Gradient descent needs $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ for every layer. Backpropagation is the chain rule, organised so that it runs from the output back to the input. Define the **error signal** of layer l as the gradient of the loss with respect to its pre-activations:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \tag{12}$$

At the output, $\boldsymbol{\delta}^{(L)} = \mathbf{p} - \mathbf{y}$ from the previous section. The pre-activation of layer $l+1$ depends on $\mathbf{z}^{(l)}$ through the activation $\sigma(\mathbf{z}^{(l)})$ followed by multiplication with $\mathbf{W}^{(l+1)}$. Applying the chain rule to these two steps gives the recursion

$$\boldsymbol{\delta}^{(l)} = \left(\mathbf{W}^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}\right) \odot \sigma'\big(\mathbf{z}^{(l)}\big), \tag{13}$$

where $\odot$ is the elementwise product. The transpose sends the error back along the same connections the signal used going forward, and the factor σ' gates it: for ReLU, no error flows back through units that were inactive.

Once $\boldsymbol{\delta}^{(l)}$ is known, the parameter gradients follow because $\mathbf{z}^{(l)}$ is linear in $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}\, \mathbf{a}^{(l-1)\top}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}. \tag{14}$$

In components, the gradient of the weight from input s to output r is $\delta_r a_s$: the error at its output times the activity at its input.

The backward pass costs about as much as the forward pass, so the gradient with respect to millions of parameters costs only about two function evaluations. This is reverse-mode automatic differentiation. PyTorch's `autograd` records the operations of the forward pass and runs this recursion automatically, so you never write it by hand. Module 4 works out what it computes for conv layers.

Generalisation

What matters is the performance on images the model has never seen. So part of the data is held back as a validation or test set and never used for training. **Overfitting** shows up as training loss that keeps falling while validation loss rises: the model is memorising the training examples instead of learning the pattern. Standard remedies are more data, data augmentation (randomly modified copies of the training images, Module 6), **weight decay** (adding $\lambda\|\theta\|^2$ to J, which pulls weights towards zero with strength λ), and dropout (Module 5).

Module 3 — Convolution math

Module 1 derived a conv layer with a single output channel that visits every pixel. This module writes down the general layer and works out the sizes and costs that follow from its settings: the kernel size K, the stride S, the padding P, and the numbers of input and output channels C_{in} and C_{out}.

Convolution or cross-correlation?

The continuous convolution $(f * g)(x) = \int f(y)\, g(x-y)\, dy$ evaluates g at a reflected argument, so the kernel is flipped. The sum in Module 1 reads the image at $i+u$, without the flip. That operation is a **cross-correlation**:

$$(X \star w)(i,j) = \sum_u \sum_v X(i+u,\, j+v)\, w(u,v). \tag{15}$$

For a learned kernel the difference does not matter: if a task needed the flipped kernel, training would simply find it. Deep-learning libraries compute the cross-correlation and call it convolution, and this course does the same.

The general conv layer

The input $\mathbf{X}$ is a tensor of shape $C_{\text{in}} \times H \times W$. The weights form a four-dimensional tensor $\mathbf{W}$ of shape $C_{\text{out}} \times C_{\text{in}} \times K \times K$, where $W_{k,c,u,v}$ is the weight that kernel k applies to input channel c at offset (u, v) inside the window. Each kernel also has a bias b_k. The output $\mathbf{Y}$ has shape $C_{\text{out}} \times H' \times W'$, and its entries are

$$Y_{k,i,j} = b_k + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} W_{k,c,u,v}\, X_{c,\, Si+u,\, Sj+v}. \tag{16}$$

In words: to get output channel k at position (i, j), place kernel k with its corner at input position (Si, Sj), multiply each of the $C_{\text{in}}K^2$ input values under it by the matching weight, sum, and add the bias. The nonlinearity is applied afterwards as a separate step.

Note that a kernel covers all input channels at once. It is a $C_{\text{in}} \times K \times K$ block of weights, so it can combine information from all colours, or from all features of the previous layer, at each position.

Stride

The stride S is the step between neighbouring kernel positions. With $S = 1$ the kernel visits every position. With $S = 2$ it visits every second row and column, so the output is about half the size in each direction. A strided convolution is therefore a learned way to downsample, an alternative to pooling (Module 5).

Padding

The kernel has to fit inside the input, so near the border there are fewer valid positions. With $S = 1$ the output has only $H - K + 1$ rows. Over many layers the image would shrink, and border pixels would influence fewer outputs than central ones. **Padding** fixes this by adding P rows of zeros at the top and bottom and P columns at the left and right, so the input becomes $(H + 2P) \times (W + 2P)$. With $S = 1$, choosing $P = (K - 1)/2$ keeps the size unchanged ("same" padding), for example $P = 1$ for $K = 3$.

Padding also connects the formula above to the centred window of Module 1. The library formula indexes the window from its top-left corner, with offsets $u, v \in \{0, \ldots, K - 1\}$. In the padded input, the window of output (i, j) (with $S = 1$) covers rows i to $i + K - 1$, which are rows $i - P$ to $i + P$ of the original image. With $P = (K - 1)/2 = r$ this is exactly the $K \times K$ neighbourhood centred on pixel (i, j). The two ways of writing the sum differ only by shifting the offset u by r.

Output size

The padded input has $H + 2P$ rows and the kernel occupies K of them. The kernel's first row can therefore start anywhere from row 0 to row $H + 2P - K$, and the starting rows are spaced S apart. Counting these positions gives the output height

$$H' = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \tag{17}$$

and the same formula with W gives the output width. Examples with $H = 32$: $K = 3$, $P = 1$, $S = 1$ gives 32 (size kept); the same with $S = 2$ gives 16 (halved); $K = 5$, $P = 0$, $S = 1$ gives 28 (shrunk by $K - 1$).

Parameters and compute

Each of the C_{out} kernels has $C_{\text{in}}K^2$ weights and one bias. Each output value takes $C_{\text{in}}K^2$ multiply-add operations, and there are $C_{\text{out}}H'W'$ output values.

Quantity	Formula	Example: $K=3$, $C_{\text{in}}=64$, $C_{\text{out}}=128$, $H'=W'=32$
Weights	$C_{\text{out}}\,C_{\text{in}}\,K^2$	73,728
Biases	C_{out}	128
Multiply-adds	$C_{\text{out}}\,C_{\text{in}}\,K^2\,H'W'$	75.5 million

The parameter count does not depend on the image size, but the compute is proportional to $H'W'$. This is why CNNs halve the resolution several times while doubling the number of channels: the compute per layer stays roughly constant while the features become more abstract.

Receptive field

The receptive field r of a unit is the width of the input region it depends on (Module 1). To compute it for a stack of layers, note that one step between neighbouring units in the input of layer l corresponds to $S_1S_2\cdots S_{l-1}$ pixels in the original image, the product of all earlier strides. The kernel of layer l spans K_l such steps, so it widens the receptive field by (K_l-1) times that product. Starting from a single pixel, the receptive field after L layers is

$$r_L = 1 + \sum_{l=1}^{L} (K_l - 1) \prod_{m=1}^{l-1} S_m. \tag{18}$$

For two stacked 3×3 layers with stride 1, $r = 1+2+2 = 5$; for three, $r = 7$. A single 7×7 layer has the same receptive field, but with C channels in and out it needs $49C^2$ weights, while three 3×3 layers need $3\cdot9C^2 = 27C^2$ and apply three nonlinearities instead of one. Small kernels stacked deep are therefore cheaper and more expressive. This is the design principle of VGG.

1×1 convolutions

With $K=1$ each output value is a linear combination of the C_{in} values at the same pixel:

$$Y_{k,i,j} = b_k + \sum_{c=1}^{C_{\text{in}}} W_{k,c}\,X_{c,i,j}. \tag{19}$$

This is a dense layer across channels, applied at every pixel separately. It is used to change the number of channels cheaply, for example to compress 256 channels to 64 before an expensive 3×3 convolution (the bottleneck blocks in ResNet and Inception).

Module 4 — Backprop through conv and pooling

Backpropagation (Module 2) needs two things from every layer: the gradient of the loss with respect to the layer's parameters, used to update them, and the gradient with respect to the layer's input, passed on to the layer below. This module computes both for a conv layer and for pooling. The result is that both gradients are again convolutions, so the backward pass has the same structure and roughly the same cost as the forward pass.

Setup

To keep the indices readable, take stride $S = 1$ and no padding. The forward pass of Module 3 is then

$$Y_{k,i,j} = b_k + \sum_{c,u,v} W_{k,c,u,v}\, X_{c,\, i+u,\, j+v}. \tag{20}$$

Backpropagation hands this layer the gradient of the loss with respect to each of its outputs, the error map

$$\delta_{k,i,j} = \frac{\partial \mathcal{L}}{\partial Y_{k,i,j}}, \tag{21}$$

which says how much the loss would change if output (k, i, j) changed. It has the same shape as $\mathbf{Y}$.

Weight and bias gradients

The weight $W_{k,c,u,v}$ appears in every output of channel k, at every position (i, j), each time multiplied by the input $X_{c,i+u,j+v}$. The chain rule sums the contributions of all outputs it affects:

$$\frac{\partial \mathcal{L}}{\partial W_{k,c,u,v}} = \sum_{i,j} \delta_{k,i,j}\, X_{c,\, i+u,\, j+v}. \tag{22}$$

This is weight sharing seen from the backward side: a shared weight collects gradient from every position where it was used. The sum has the form of a cross-correlation: input channel c is correlated with error map k, and the result is evaluated at the offset (u, v). It gives a $K \times K$ array, one value per weight of the kernel.

The bias b_k is added to every output of channel k with factor 1, so its gradient is the sum of the error map over all positions:

$$\frac{\partial \mathcal{L}}{\partial b_k} = \sum_{i,j} \delta_{k,i,j}. \tag{23}$$

Input gradient

To pass the error on to the layer below, we need the gradient with respect to each input value. The input $X_{c,p,q}$ is used by every output (k, i, j) whose window covers it, that is with $i + u = p$ and $j + v = q$, and each time it is multiplied by $W_{k,c,u,v}$. Substituting $i = p - u$ and $j = q - v$ and summing over all of them:

$$\frac{\partial \mathcal{L}}{\partial X_{c,p,q}} = \sum_{k=1}^{C_{\text{out}}} \sum_{u,v} \delta_{k,\, p-u,\, q-v}\, W_{k,c,u,v}. \tag{24}$$

The error map is now read at $p - u$, so the kernel is applied flipped by 180°: this is a true convolution in the sense of Module 3, of each error map with its kernel, summed over the output channels k. Terms where $p - u$ or $q - v$ fall outside the error map are zero. This is the same as padding the error map with $K - 1$ zeros on each side, which is why the result has the full input size $H \times W$ again.

There is also a linear-algebra view. Without the bias the conv layer is linear in its input, so after flattening the tensors it can be written as $\mathbf{Y} = \mathbf{M}\mathbf{X}$ with a large sparse matrix $\mathbf{M}$ whose rows contain shifted copies of the kernel weights (a block-Toeplitz matrix). Exactly as for the dense layer in Module 2, the input gradient is $\mathbf{M}^\top$ applied to the error. Applying $\mathbf{M}^\top$ is called a

transposed convolution; the same operation is used as an upsampling layer in segmentation and image-generation networks.

With stride $S > 1$ the layer computes only every S-th output, so the error map has gaps relative to the input grid. Inserting $S - 1$ zeros between neighbouring entries of the error map restores the spacing, and then the same formula applies.

Nonlinearities and pooling

Layers without weights follow the same rule: the error is passed back multiplied by the local derivative of the operation.

- **ReLU.** The derivative is 1 where the pre-activation was positive and 0 elsewhere, so the error passes through active units unchanged and is blocked at inactive ones.
- **Max pooling.** It splits each feature map into windows, usually 2×2 with stride 2, and keeps the largest value of each. The output depends only on that largest input, with derivative 1, and not on the others. So the whole error goes back to the position of the maximum, which is remembered from the forward pass, and the other positions get zero.
- **Average pooling.** It outputs the mean of a $K \times K$ window, so every input has derivative $1/K^2$, and the error is divided equally among them.
- **Global average pooling.** It averages each channel over all $H \times W$ positions to a single number, so each position receives the error divided by HW.

Pooling has no parameters. It reduces the spatial size, which saves compute and enlarges the receptive field of later layers, and it makes the output less sensitive to small shifts: the maximum of a 2×2 window does not change if the strongest feature moves within that window.

Check yourself

PyTorch computes all of this automatically, but deriving it once by hand is the best test of understanding. Implement the forward sum with NumPy loops for a small random input, compute the weight, bias and input gradients from the formulas above, and compare them with the gradients PyTorch returns for `torch.nn.functional.conv2d` on the same tensors created with `requires_grad=True`. They should agree to about 10^{-6}.

Module 5 — Building blocks and architectures

Modules 1–4 cover the conv layer itself. A network that trains well needs a few more components, mostly to keep signals and gradients at a healthy scale through many layers. This module introduces them and then shows how they are arranged in the architectures that shaped the field.

Batch normalisation

As training updates the early layers, the scale of the values reaching later layers keeps changing. If activations grow or shrink from layer to layer, gradients explode or vanish, and training only works with small learning rates. **Batch normalisation** (BN) rescales each channel to a controlled mean and variance.

For one channel, BN computes the mean μ_B and variance σ_B^2 of all its values in the current mini-batch, over all images and all positions. Each value x is normalised with these, and then a learned scale γ and shift β are applied:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta. \tag{25}$$

The small constant ϵ (e.g. 10^{-5}) prevents division by zero. Because γ and β are learned per channel, the network can undo the normalisation where that helps. β plays the role of the bias of the preceding conv layer, which is why a conv followed by BN is usually created without its own bias.

In practice BN allows larger learning rates, speeds up training considerably, and regularises slightly, because the batch statistics add some noise.

At test time there is often a single image and no meaningful batch. So during training BN also keeps running averages of μ_B and σ_B^2, and uses those at test time. In PyTorch, `model.train()` and `model.eval()` switch between the two behaviours. Forgetting `model.eval()` before evaluating is a common bug.

Dropout

During training, **dropout** sets each activation to zero with probability p (e.g. 0.3), independently for every example. The surviving activations are multiplied by $1/(1-p)$ so that their expected value is unchanged. The network cannot rely on any single unit and has to spread information across many, which reduces overfitting. At test time dropout is switched off. In CNNs it is mainly used in the dense layers at the end; the conv layers have few parameters and are already regularised by BN.

Residual connections

Making a plain CNN deeper than about 20 layers made it worse, even on the training data. This is an optimisation problem, not overfitting: the error signal passes through every layer on its way back and is multiplied by each layer's Jacobian (the recursion of Module 2), so over many layers it tends to shrink or grow exponentially.

ResNet (2015) solves this with residual blocks. A block computes a correction $F(\mathbf{x}; \theta)$, typically two conv–BN–ReLU layers, and adds it to its own input:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}; \theta), \qquad \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}}. \tag{26}$$

The identity term I means the error passes back through the block unchanged, plus the contribution of F, so there is always a direct path to the early layers. In addition, a block that is not needed can learn $F \approx 0$ and becomes the identity, so adding blocks rarely makes the network worse. This made networks with 50 to over 150 layers trainable. When a block changes the number of channels or the resolution, the skip path uses a 1×1 convolution with the matching stride so that the shapes of $\mathbf{x}$ and $F(\mathbf{x})$ agree.

In the continuous picture, the update $\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l)$ is one forward-Euler step of the differential equation $d\mathbf{x}/dt = F(\mathbf{x}, t)$. A deep ResNet discretises a flow on feature space, an observation that led to neural ODEs.

Weight initialisation

Before training, the weights are random, and their scale matters. Each pre-activation is a sum of $n_{\text{in}} = C_{\text{in}}K^2$ products of a weight and an input. If the weights are independent with variance $\text{Var}(W)$, the variance of the pre-activation is about $n_{\text{in}}\,\text{Var}(W)$ times the mean square of the inputs, and ReLU halves that mean square because it zeros half the values. Keeping the variance the same from layer to layer therefore requires

$$\text{Var}(W) = \frac{2}{n_{\text{in}}}, \tag{27}$$

which is **He** (or Kaiming) initialisation, $W \sim \mathcal{N}(0,\, 2/n_{\text{in}})$. With smaller weights the signal decays to zero over depth; with larger ones it explodes. PyTorch's default initialisation is a variant of this, and with BN after every conv the exact scale matters much less.

Landmark architectures

The history of CNNs is mostly a sequence of ways to go deeper while staying trainable.

Year	Network	What it introduced	Layers
1998	LeNet-5	Conv + pooling + dense layers; handwritten digit recognition	5
2012	AlexNet	ReLU, dropout, GPU training; won ImageNet by a wide margin	8
2014	VGG	Only 3×3 kernels, stacked deep (Module 3)	16–19
2015	ResNet	Residual connections and batch norm	18–152
2019	EfficientNet	Scaling depth, width and resolution together	varies

Vision transformers have since matched or beaten CNNs when training data is very large. CNNs remain the better choice for small and medium datasets, for embedded hardware, and for grid data where local structure dominates.

The canonical layout

Most CNN classifiers, including the one in Module 6, follow the same three-part layout:

1. **Stem.** A first conv layer lifts the 3 colour channels to a larger number of feature channels, e.g. 32 or 64.
2. **Stages.** Each stage is a few conv–BN–ReLU blocks (or residual blocks) at a fixed resolution. Between stages the height and width are halved, by max pooling or a stride-2 conv, and the number of channels is doubled. As shown in Module 3, this keeps the compute per stage roughly constant while the receptive field grows.
3. **Head.** Global average pooling reduces the final $C \times H \times W$ tensor to C numbers, one average per channel. This makes the output invariant to where in the image a feature appeared (Module 1) and independent of the input size. A linear layer then maps the C numbers to the class logits.

Module 6 — Application: training a CNN in PyTorch

The script below trains a 1.15 M-parameter, VGG-style CNN on CIFAR-10 (60,000 colour images, 32×32, 10 classes) and typically reaches about 90% test accuracy in 30 epochs. Every piece maps to a module above.

Setup

```
python -m venv cnn && source cnn/bin/activate
pip install torch torchvision
python train_cnn.py
```

The first run downloads CIFAR-10 (∼170 MB) into `./data`. On a recent GPU an epoch takes seconds; on a CPU, expect minutes per epoch, so start with `EPOCHS = 5` to check everything runs.

Tensor shapes through the network

Stage	Layers	Output ($C \times H \times W$)	Parameters
Input	normalised RGB image	$3 \times 32 \times 32$	0
Stage 1	2×(conv 3×3, BN, ReLU), max pool 2×2	$64 \times 16 \times 16$	38,848
Stage 2	2×(conv 3×3, BN, ReLU), max pool 2×2	$128 \times 8 \times 8$	221,696
Stage 3	2×(conv 3×3, BN, ReLU), max pool 2×2	$256 \times 4 \times 4$	885,760
Head	global average pool, dropout, linear	10 logits	2,570

Every conv layer uses $K = 3$, $P = 1$ and $S = 1$, so by the output-size formula of Module 3 it keeps the height and width. Only the max-pooling layers change the size, each halving it, while the number of channels doubles from stage to stage: the canonical layout of Module 5. Applying the receptive-field formula to the whole stack (six 3×3 convs and three stride-2 poolings) gives $r = 36$, so each unit of the last stage depends on the entire 32×32 image. The parameter column follows from $C_{\text{out}} C_{\text{in}} K^2$ per conv plus $2C_{\text{out}}$ per BN layer (γ and β).

The full script (`train_cnn.py`)

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# ---- 1. Device and hyperparameters ----
device = ("cuda" if torch.cuda.is_available()
          else "mps" if torch.backends.mps.is_available() else "cpu")
EPOCHS, BATCH, LR, WD = 30, 128, 0.1, 5e-4
torch.manual_seed(0)

# ---- 2. Data: normalise per channel, augment the training set ----
mean, std = (0.4914, 0.4822, 0.4465), (0.2470, 0.2435, 0.2616)
train_tf = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
test_tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize(mean, std)])

train_set = datasets.CIFAR10("data", train=True, download=True, transform=train_tf)
test_set = datasets.CIFAR10("data", train=False, download=True, transform=test_tf)
train_loader = DataLoader(train_set, BATCH, shuffle=True, num_workers=2, drop_last=True)
test_loader = DataLoader(test_set, 256, shuffle=False, num_workers=2)
```

```
# ---- 3. Model: 3 stages (32 -> 16 -> 8 -> 4) -> global pool -> linear ----
def conv_bn_relu(c_in, c_out):
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, kernel_size=3, padding=1, bias=False),  # BN supplies the bias
        nn.BatchNorm2d(c_out),
        nn.ReLU(inplace=True),
    )

class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            conv_bn_relu(3, 64),   conv_bn_relu(64, 64),    nn.MaxPool2d(2),  # 64 x 16 x 16
            conv_bn_relu(64, 128), conv_bn_relu(128, 128),  nn.MaxPool2d(2),  # 128 x 8 x 8
            conv_bn_relu(128, 256), conv_bn_relu(256, 256), nn.MaxPool2d(2),  # 256 x 4 x 4
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),  # global average pooling -> 256 x 1 x 1
            nn.Flatten(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes),  # logits
        )

    def forward(self, x):
        return self.head(self.features(x))

model = SmallCNN().to(device)
print(f"parameters: {sum(p.numel() for p in model.parameters()):,}")

# ---- 4. Loss, optimiser, learning-rate schedule ----
criterion = nn.CrossEntropyLoss()  # softmax + NLL; gradient w.r.t. logits is p - y
optimizer = torch.optim.SGD(model.parameters(), lr=LR, momentum=0.9,
                            weight_decay=WD, nesterov=True)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=LR, epochs=EPOCHS, steps_per_epoch=len(train_loader))

# ---- 5. Training and evaluation loops ----
def train_one_epoch():
    model.train()                       # BN uses batch stats, dropout active
    total, correct, loss_sum = 0, 0, 0.0
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)               # forward pass
        loss = criterion(logits, y)
        optimizer.zero_grad()
        loss.backward()                 # backprop: fills p.grad for every parameter
        optimizer.step()                # theta <- theta - lr * grad (with momentum)
        scheduler.step()
        loss_sum += loss.item() * y.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        total += y.size(0)
    return loss_sum / total, correct / total

@torch.no_grad()
def evaluate():
    model.eval()                        # BN uses running stats, dropout off
    total, correct = 0, 0
    for x, y in test_loader:
        x, y = x.to(device), y.to(device)
        correct += (model(x).argmax(1) == y).sum().item()
        total += y.size(0)
    return correct / total
```

```
if __name__ == "__main__":
    for epoch in range(1, EPOCHS + 1):
        tr_loss, tr_acc = train_one_epoch()
        te_acc = evaluate()
        print(f"epoch {epoch:2d}  loss {tr_loss:.3f}  train {tr_acc:.3f}  test {te_acc:.3f}")
    torch.save(model.state_dict(), "cnn_cifar10.pt")
```

What each part does

1. **Device.** The script uses an NVIDIA GPU (`cuda`) or an Apple-silicon GPU (`mps`) if one is present, otherwise the CPU. `.to(device)` moves the model and each batch there.
2. **Data.** `Normalize` subtracts the per-channel mean of the CIFAR-10 training set and divides by its standard deviation, so the inputs have mean 0 and variance 1, the scale the initialisation of Module 5 assumes. `RandomCrop` (with 4 pixels of padding) and `RandomHorizontalFlip` are **data augmentation**: every epoch the network sees slightly shifted and mirrored copies of each image. This adds the translation and mirror symmetries directly to the data and is the cheapest defence against overfitting. The test set is only normalised, never augmented. A `DataLoader` shuffles the data and serves it in mini-batches of `BATCH` images.
3. **Model.** `conv_bn_relu` builds one conv–BN–ReLU block. The conv has `bias=False` because the BN shift β takes over that role (Module 5). `SmallCNN` stacks six such blocks in three stages with max pooling in between, followed by the head: global average pooling, dropout, and a linear layer to the 10 logits. `forward` defines how the input flows through these parts; PyTorch derives the backward pass from it.
4. **Optimisation.** `CrossEntropyLoss` applies the softmax and the cross-entropy of Module 2 in one numerically stable step, so the model outputs raw logits. The optimiser is SGD with Nesterov momentum 0.9. `weight_decay` adds the $\lambda\|\theta\|^2$ penalty of Module 2 with $\lambda = 5 \cdot 10^{-4}$. The one-cycle schedule raises the learning rate from a small value to 0.1 during the first part of training and then lowers it to nearly zero, which allows large steps early and fine adjustment at the end.
5. **Training loop.** For each batch, `train_one_epoch` runs the five steps of gradient descent: forward pass (`model(x)`), loss, clearing the gradients left from the previous batch (`zero_grad`), backpropagation (`loss.backward()`, which fills `p.grad` for every parameter `p`), and the update (`optimizer.step()`). `model.train()` at the start switches BN to batch statistics and turns dropout on; `model.eval()` in `evaluate` switches BN to its running averages and turns dropout off. `torch.no_grad()` skips gradient bookkeeping during evaluation, which saves memory and time.

Reading the output

Each epoch prints the mean training loss, the training accuracy and the test accuracy. The training loss should start around 1.8 (the loss of a uniform guess over 10 classes is $\ln 10 \approx 2.3$) and fall steadily. Training accuracy is usually somewhat above test accuracy; if training accuracy approaches 100% while test accuracy stops improving, the model is overfitting, and more augmentation, more weight decay or a smaller model will help. The `if __name__ == "__main__"` guard is needed on macOS and Windows, where `num_workers > 0` starts the data-loading workers as new processes that re-import the script.

Using the trained model

```
model = SmallCNN()
model.load_state_dict(torch.load("cnn_cifar10.pt"))
model.eval()
with torch.no_grad():
    probs = model(x.unsqueeze(0)).softmax(1)  # x: a normalised 3 x 32 x 32 tensor
```

Exercises and next steps

Each exercise tests one module; together they take a few evenings.

- ☐ **Module 3:** compute by hand the output shape and parameter count of `nn.Conv2d(64, 32, kernel_size=5, stride=2, padding=1)` on a $64 \times 31 \times 31$ input, then confirm in PyTorch.
- ☐ **Module 4:** implement conv2d forward and its three gradients in NumPy and check them against `torch.autograd` (see "Check yourself").
- ☐ **Module 5:** replace the plain blocks with residual blocks, $\mathbf{y} = \mathrm{ReLU}(\mathbf{x} + F(\mathbf{x}))$, using a 1×1 conv on the skip path when the channel count changes. Compare test accuracy.
- ☐ **Module 6:** train once without augmentation and plot train and test accuracy per epoch. Where does the gap open?
- ☐ **Module 6:** visualise the 64 first-layer filters (`model.features[0][0].weight`) as 3×3 RGB patches. You should see edge and colour detectors.

Where to go next: transfer learning (fine-tune a pretrained `torchvision.models.resnet18` on your own images, usually the most practical route with little data); object detection and segmentation (YOLO, U-Net); and equivariant CNNs for rotations and other groups, the natural continuation of the symmetry view from Module 1. For embedded deployment, export with `torch.onnx.export` and run it from C++ through ONNX Runtime or TensorRT.